Here is the definitive, all-in-one Master Deployment Guide. Every script, configuration, and application file is compiled here so you can copy and paste everything directly into your production environment.

The Directory Structure

Create this exact folder layout on your 16GB DigitalOcean Production Droplet:
Bash
mkdir -p /opt/ev_platform/valhalla_data
mkdir -p /opt/ev_platform/nginx
mkdir -p /opt/ev_platform/fastsim-python
mkdir -p /opt/ev_platform/nodejs-api

Database Configuration (Run in Supabase SQL Editor)

Copy and run this entire block in your Supabase SQL Editor to prepare your PostGIS spatial engine, connection pooler, and automated maintenance.
SQL

Enable Core Extensions

```
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS pg_cron;
```

Optimize Autovacuum for High-Frequency Charger Updates

```
ALTER TABLE public.stations SET (autovacuum_vacuum_scale_factor = 0.05, autovacuum_vacuum_threshold = 50);
ALTER TABLE public.charger_units SET (autovacuum_vacuum_scale_factor = 0.05, autovacuum_vacuum_threshold = 50);
```

Create the High-Speed Spatial Index

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_stations_geom_gist ON public.stations USING GIST(geom);
```

Deploy the Spatial Interception Engine

```
CREATE OR REPLACE FUNCTION find_nearest_chargers(
  deplete_lat double precision,
```

```
  deplete_lng double precision,
  search_radius_meters integer
)
RETURNS TABLE (
  id varchar,
  name varchar,
  lat numeric,
  lng numeric,
  power_kw numeric,
  distance_meters float
) AS $$
BEGIN
  RETURN QUERY
  SELECT
    s.id, s.name, s.lat, s.lng, s.power_kw,
    ST_Distance(s.geom, ST_SetSRID(ST_MakePoint(deplete_lng, deplete_lat),
4326)::geography) AS distance_meters
  FROM public.stations s
  WHERE ST_DWithin(s.geom, ST_SetSRID(ST_MakePoint(deplete_lng, deplete_lat),
4326)::geography, search_radius_meters)
    AND s.status = 'Active'
    AND s.is_deleted = false
    AND 'CCS2' = ANY(s.connector_types)
  ORDER BY s.power_kw DESC, distance_meters ASC
  LIMIT 1;
END;
$$ LANGUAGE plpgsql;
```

Schedule Automated Log Cleanup (Runs daily at 2:00 AM)

```
SELECT cron.schedule('purge_historical_logs', '0 2 * * *', $$
  DELETE FROM public.visitor_logs WHERE created_at < NOW() - INTERVAL '30 days';
$$);
```

Map Compilation (Run on Temporary 64GB Burner Droplet)

Execute these commands via SSH on your temporary high-RAM server.
Bash
```
mkdir -p /root/valhalla_compile/custom_files
cd /root/valhalla_compile
```

```
# Download OSM Data for India
wget https://download.geofabrik.de/asia/india-latest.osm.pbf -O
custom_files/india-latest.osm.pbf

# Run the Compiler (Takes 2-4 hours)
docker run -it --name valhalla_builder \
  -v $PWD/custom_files:/custom_files \
  -e build_elevation=True \
  -e build_admins=True \
  -e build_time_zones=True \
  ghcr.io/valhalla/valhalla-scripted:latest

# Compress the Output Map
tar -czvf valhalla_india_graph.tar.gz -C custom_files valhalla_tiles valhalla.json

# (Download valhalla_india_graph.tar.gz to your Production Droplet, extract into
/opt/ev_platform/valhalla_data/, then DESTROY the Burner server)
```

Production Environment Files (Save on 16GB Production Droplet)

File 1: /opt/ev_platform/docker-compose.yml

YAML

```
version: '3.8'
services:
  nginx:
    image: nginx:alpine
    container_name: ev_nginx_ingress
    restart: always
    ports:
      - "80:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on:
      - orchestrator

  orchestrator:
    build: ./nodejs-api
    container_name: ev_orchestrator_core
    restart: always
    expose:
```

```
    - "3000"
  environment:
    - NODE_ENV=production
    - SUPABASE_POOL_URL=postgres://[DB_USER]:[DB_PASS]@[DB_HOST]:6543/postgres
    - VALHALLA_URL=http://valhalla:8002
    - FASTSIM_URL=http://fastsim:8000
  depends_on:
    - valhalla
    - fastsim

fastsim:
  build: ./fastsim-python
  container_name: ev_fastsim_physics
  restart: always
  expose:
    - "8000"
  deploy:
    resources:
      limits:
        cpus: '2.0'

valhalla:
  image: ghcr.io/valhalla/valhalla:latest
  container_name: ev_valhalla_geometry
  restart: always
  expose:
    - "8002"
  volumes:
    - ./valhalla_data:/custom_files
  command: valhalla_service /custom_files/valhalla.json 1
```

File 2: /opt/ev_platform/nginx/nginx.conf

```
Nginx
worker_processes auto;
events { worker_connections 1024; }

http {
    include mime.types;
    default_type application/octet-stream;

    add_header X-Frame-Options DENY;
    add_header X-Content-Type-Options nosniff;
    add_header X-XSS-Protection "1; mode=block";
```

```
    gzip on;
    gzip_types application/json text/plain;
    gzip_min_length 1000;

    limit_req_zone $binary_remote_addr zone=api_limit_zone:10m rate=10r/s;

    server {
        listen 80;
        server_name api.yourdomain.com; # Update to your Cloudflare domain

        location /api/v1/route/ {
            limit_req zone=api_limit_zone burst=20 nodelay;
            proxy_pass http://orchestrator:3000;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        }
        location / { return 404; }
    }
}
```

File 3: /opt/ev_platform/fastsim-python/main.py

Python

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import fastsim
import numpy as np
from functools import lru_cache

app = FastAPI()

class SimulationPayload(BaseModel):
    vehicle_specs: dict
    initial_soc: float
    route_coordinates: list
    route_distance_km: float
    route_time_sec: float

@lru_cache(maxsize=100)
def get_cached_vehicle(vehicle_model_id: str, specs_tuple: tuple):
    return fastsim.vehicle.Vehicle.from_dict(dict(specs_tuple))
```

```
@app.post("/simulate")
def simulate_ev_route(payload: SimulationPayload):
    try:
        specs_tuple = tuple(payload.vehicle_specs.items())
        veh = get_cached_vehicle("dynamic_model", specs_tuple)

        time_array = np.linspace(0, payload.route_time_sec, len(payload.route_coordinates))
        speed_array = np.full(len(payload.route_coordinates), payload.route_distance_km /
(payload.route_time_sec / 3600))

        cyc = fastsim.cycle.Cycle.from_dict({"cycSecs": time_array, "cycMps": speed_array *
(1000/3600)})
        sim = fastsim.simulator.SimDrive(cyc, veh)
        sim.sim_drive(payload.initial_soc / 100.0)

        return {"status": "success", "soc_array": (sim.soc * 100).tolist()}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

File 4: /opt/ev_platform/fastsim-python/requirements.txt

```
Plaintext
fastapi==0.103.1
uvicorn==0.23.2
gunicorn==21.2.0
fastsim==2.1.2
numpy==1.25.2
pydantic==2.3.0
```

File 5: /opt/ev_platform/fastsim-python/Dockerfile

```
Dockerfile
FROM python:3.10-slim
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends gcc g++ && rm -rf
/var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["gunicorn", "main:app", "-w", "4", "-k", "uvicorn.workers.UvicornWorker", "-b",
"0.0.0.0:8000"]
```

File 6: /opt/ev_platform/nodejs-api/Dockerfile

Dockerfile
```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

Final Launch Execution (Run on Production Droplet)

With all files securely in place, run these commands to compile the images, launch the Docker network, and bring your engine online.

Bash
```
cd /opt/ev_platform

# Launch the entire microservice architecture
docker-compose up -d --build

# Verify network status (All containers should say "Up")
docker-compose ps
```